Отчёт по комплексному циклу практических работ (№1–7)

Тема: Разработка веб-приложения «Менеджер задач» (Task Manager) на языке Python с использованием микрофреймворка Flask и встраиваемой СУБД SQLite.

> **Цель проекта:** Изучение полного цикла создания информационных систем: проектирование реляционной схемы данных (DDL), реализация CRUD-операций в консоли, разработка маршрутов веб-сервера на Flask, рендеринг динамических страниц шаблонизатором Jinja2, защита от SQL-инъекций и управление транзакциями по отказоустойчивому паттерну `try/except/finally/rollback`.

1. Используемый стек технологий и архитектура

Компонент	Инструмент	Назначение и роль в проекте
Язык разработки	Python 3.8+	Основной язык серверной логики, обработки HTTP-запросов и манипуляции данными.
База данных	SQLite3 (модуль `sqlite3`)	Встраиваемая бессерверная реляционная СУБД; хранение схемы и записей в файле `app.db`.
Веб-фреймворк	Flask (WSGI)	Маршрутизация (routing), обработка запросов GET/POST, контекст `request`, редиректы.
Шаблонизатор	Jinja2	Генерация HTML-документов, внедрение серверных переменных, циклы `for`, условия `if`.
Интерфейс	HTML5 / CSS3	Пользовательский UI со списками задач, формами и визуальным отображением статусов.

2. Структура файлов готового проекта

```
task_manager/
├── app.db              # Файл базы данных SQLite (содержит таблицу tasks)
├── init_db.py          # Скрипт создания структуры БД (DDL)
├── check_db.py         # Проверочный скрипт метаданных таблицы
├── crud_console.py     # Консольный CRUD-интерфейс (прототип)
├── app.py              # Основное веб-приложение Flask с защищёнными транзакциями
└── templates/
    └── index.html      # Шаблон интерфейса со стилями и формами действий
```

3. Схема базы данных (DDL)

Таблица `tasks` спроектирована в первой практической работе со следующими спецификациями:

- `id INTEGER PRIMARY KEY` — уникальный суррогатный ключ строки (алиас для `rowid` с автоматической инкрементацией).
- `title TEXT NOT NULL` — обязательное строковое поле названия задачи.
- `status TEXT DEFAULT 'new'` — статус исполнения ('new' по умолчанию, 'done' при завершении).

4. Этапы практических работ и прогресс разработки

Этап	Название работы	Ключевые изученные концепции и результат
Работа №1	Создание БД и таблицы	DDL-запросы, `connect()`, курсор, необходимость `commit()` для записи на диск, системная таблица `sqlite_master`.
Работа №2	Консольный CRUD	Параметризованные запросы с плейсхолдерами `?` (защита от SQL-инъекций), `lastrowid`, `rowcount`, безопасный парсинг ввода.
Работа №3	Первый сервер Flask	WSGI-сервер, порт 5000, декоратор `@app.route`, view-функции, режим отладки `debug=True`, обработка ошибки 404.
Работа №4	Интеграция с Jinja2	Переход от HTML в строках к шаблонам, фабрика строк `conn.row_factory = sqlite3.Row`, рендеринг списков через `{% for %}`.
Работа №5	POST-запросы и PRG	HTML-формы, чтение `request.form`, паттерн Post/Redirect/Get (HTTP 302) для предотвращения дублирования записей при нажатии F5.
Работа №6	Защита транзакций	Архитектурный шаблон `try/except/finally`: вызов `conn.rollback()` при ошибках целостности и гарантированный `close()`.
Работа №7	Финальный мини-проект	Полный браузерный CRUD, параметры маршрутов `<int:task_id>`, переключение статусов через SQL-конструкцию `CASE WHEN`.

5. Исходный код ключевых модулей

5.1. Скрипт инициализации БД (`init_db.py`)

```
import sqlite3

conn = sqlite3.connect('app.db')
cursor = conn.cursor()

cursor.execute("""
    CREATE TABLE IF NOT EXISTS tasks (
        id     INTEGER PRIMARY KEY,
        title  TEXT NOT NULL,
        status TEXT DEFAULT 'new'
    )
""")

conn.commit()
conn.close()
print("База данных и таблица tasks успешно созданы.")
```

5.2. Консольный клиент (`crud_console.py`, фрагмент логики)

```
def add_task():
    title = input("Название задачи: ").strip()
    if not title:
        print("⚠ Название не может быть пустым.")
        return
    conn = get_connection()
    cursor = conn.cursor()
    # Использование параметризованного запроса предотвращает SQL-инъекции
    cursor.execute("INSERT INTO tasks (title) VALUES (?)", (title,))
    conn.commit()
    print(f"✅ Задача добавлена. ID = {cursor.lastrowid}")
    conn.close()
```

5.3. Итоговое веб-приложение (`app.py`)

```
import sqlite3
from flask import Flask, render_template, request, redirect, url_for

app = Flask(__name__)
DB_PATH = 'app.db'

def get_connection():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn

def fetch_all_tasks():
    conn = get_connection()
    try:
        cursor = conn.cursor()
        cursor.execute("SELECT id, title, status FROM tasks ORDER BY id")
        return cursor.fetchall()
    except sqlite3.Error as e:
        print(f"[DB ERROR fetch] {e}")
        return []
    finally:
        conn.close()

def insert_task(title):
    conn = get_connection()
    try:
        cursor = conn.cursor()
        cursor.execute("INSERT INTO tasks (title) VALUES (?)", (title,))
        conn.commit()
        return True, None
    except sqlite3.Error as e:
        conn.rollback()
        print(f"[DB ERROR insert] {e}")
        return False, "Не удалось сохранить задачу."
    finally:
        conn.close()

def toggle_task_status(task_id):
    conn = get_connection()
    try:
        cursor = conn.cursor()
        cursor.execute("""
            UPDATE tasks
            SET status = CASE WHEN status = 'done' THEN 'new' ELSE 'done' END
            WHERE id = ?
        """, (task_id,))
        conn.commit()
        return cursor.rowcount
    except sqlite3.Error as e:
        conn.rollback()
        print(f"[DB ERROR toggle] {e}")
        return 0
    finally:
        conn.close()

def delete_task_by_id(task_id):
    conn = get_connection()
    try:
        cursor = conn.cursor()
        cursor.execute("DELETE FROM tasks WHERE id = ?", (task_id,))
        conn.commit()
```

6. Примеры работы и тестовые сценарии

6.1. Проверка паттерна Post/Redirect/Get (PRG)

В ходе практических работ проверена корректность работы HTTP-запросов через вкладку браузера Network:

- Отправка формы добавления задачи генерирует запрос: `POST /` с кодом ответа `302 Found` (заголовок `Location: /`).
- Браузер выполняет последующий запрос: `GET /` с кодом `200 OK`.
- **Результат:** нажатие клавиши F5 пользователем повторно отправляет безопасный GET-запрос, исключая дублирование строк в базе данных.

6.2. Проверка устойчивости к SQL-инъекциям

В поле ввода названия задачи передавалась строка: `'); DROP TABLE tasks; --`

- Благодаря плейсхолдеру `?` драйвер передал строку целиком как параметр текстового значения (литерал).
- **Результат:** в базе успешно создана запись с указанным текстом; структура таблицы `tasks` осталась неповреждённой.

6.3. Проверка транзакционной безопасности и отсутствия блокировок

Были проведены нагрузочные тесты с искусственным вызовом `sqlite3.IntegrityError` (передача `None` в поле с ограничением `NOT NULL`):

- Блок `except` выполнил `conn.rollback()`, предотвратив появление повреждённых данных в файле.
- Блок `finally` гарантированно вызвал `conn.close()`, сняв блокировку файла БД.
- Параллельные обращения из консольного клиента `crud_console.py` выполнялись без ошибок вида `database is locked`.

7. Ответы на ключевые контрольные вопросы для защиты

Контрольный вопрос	Ответ для защиты
Зачем нужен `conn.commit()` и что будет без него?	В SQLite транзакция открывается автоматически. Метод `commit()` выполняет сброс данных из буфера памяти на физический диск. Без вызова `commit()` при закрытии соединения происходит неявный откат (ROLLBACK), и данные в файле не сохраняются.
В чём разница между GET и POST?	GET предназначен для безопасного чтения ресурсов без изменения состояния сервера (идемпотентный метод, параметры передаются в URL). POST используется для передачи данных в теле запроса для изменения состояния сервера (создание, удаление, обновление).
Зачем нужен конвертер `<int:task_id>` в маршрутах?	Он автоматически парсит параметр URL, валидирует его тип как целое число и преобразует в тип `int` в аргументах функции. Если в URL передан нечисловой фрагмент (например, `/delete/abc`), Flask самостоятельно генерирует ответ `404 Not Found`.
Что означает `cursor.rowcount = 0` при UPDATE или DELETE?	Это означает, что запрос синтаксически выполнился успешно, но условию `WHERE id = ?` не соответствовала ни одна запись (запрашиваемый ID не найден в БД).

Контрольный вопрос	Ответ для защиты
Зачем использовать `sqlite3.Row`?	Обеспечивает гибридный доступ к полям строки как по числовым индексам, так и по символьным именам столбцов. Это критически необходимо для обращения к свойствам в шаблонах Jinja2 вида `{{ item.title }}`.

8. Выводы

В ходе выполнения лабораторных работ №1–7 был успешно освоен базовый стек бэкенд-разработки на Python. Создано законченное веб-приложение со списком задач, реализующее архитектуру разделения данных, бизнес-логики и представления, а также отвечающее требованиям надежности и безопасности баз данных.